

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота 4

з дисципліни
«Об'єктно-орієнтоване програмування»

на тему

« Вдосконалення структури коду графічного редактора об'єктів »

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім'ям Lab4.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налогодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіант завдання

Номер варіанту завдання дорівнює номеру зі списку студентів у журналі: $\mathbf{Ж = 3}$.

1. Вимоги до об'єктно-орієнтованої архітектури редактора:

- **Форма створення об'єкта MyEditor:** Оскільки номер у журналі непарний ($\mathbf{Ж = 3}$), програмується **глобальний статичний об'єкт класу MyEditor**. Пам'ять під нього виділяється компілятором у сегменті статичних даних на початку роботи процесу, а знищення відбувається автоматично при завершенні програми.
- **Сховище об'єктів:** Динамічний масив вказівників `Shape** pshape` обсягом $\mathbf{N = Ж + 100 = 3 + 100 = 103}$ об'єкти (оскільки $3 \bmod 3 = 0$).
- **"Гумовий" слід при вводі об'єктів:** Для всіх фігур малюється **пунктирною лінією чорного кольору (PS_DOT)**.

2. Вимоги до геометричних форм та їх відображення:

Окрім чотирьох базових типів фігур, запрограмовано введення та відображення двох нових фігур із використанням **множинного віртуального успадкування**:

1. **Крапка (PointShape):** жирна точка радіусом 2 пікселі.
2. **Лінія (LineShape):** відрізок між точками натискання та відпускання миші.
3. **Прямокутник (RectShape):** ввід від центру до одного з кутів ($3 \bmod 2 = 1$), чорний контур без заповнення ($3 \bmod 5 = 3$).
4. **Еліпс (EllipseShape):** ввід по двох протилежних кутах описаного прямокутника ($3 \bmod 2 = 1$), контур із кольоровим (рожевим) заповненням ($3 \bmod 5 = 3, 3 \bmod 6 = 3$).
5. **Лінія з кружечками (LineOOShape):** нова складена фігура, реалізована множинним успадкуванням від `LineShape` та `EllipseShape`. Метод `Show` повторно викликає `LineShape::Show` для малювання відрізка та `EllipseShape::Show` для малювання двох кружечків на кінцях.
6. **Каркас куба (CubeShape):** нова просторова фігура, реалізована множинним успадкуванням від `RectShape` та `LineShape`. Метод `Show` викликає

RectShape::Show для передньої та задньої граней і LineShape::Show для 4 з'єднувальних бічних ребер.

3. Вимоги до інтерфейсу користувача (Toolbar та меню):

- На панелі **Toolbar** розташовано **6 кнопок** із графічними піктограмами відповідних інструментів.
- Усі кнопки Toolbar забезпечені україномовними **спливаючими підказками (Tooltips)**.
- Меню **"Об'єкти"** містить 6 підпунктів, розташовується праворуч від "Файл" і ліворуч від "Довідка", та синхронізоване з кнопками Toolbar (активний пункт позначається галочкою через OnInitMenuPopup).

Код вихідних файлів:

shape.h:

#pragma once

#include <windows.h>

class Shape {

protected:

long xs1, ys1, xs2, ys2;

public:

virtual ~Shape() {}

void Set(long x1, long y1, long x2, long y2);

virtual void Show(HDC hdc) = 0;

};

class PointShape : virtual public Shape {

public:

void Show(HDC hdc) override;

};

class LineShape : virtual public Shape {

public:

void Show(HDC hdc) override;

};

class RectShape : virtual public Shape {

public:

void Show(HDC hdc) override;

};

class EllipseShape : virtual public Shape {

public:

void Show(HDC hdc) override;

};

class LineOOShape : public LineShape, public EllipseShape {

public:

void Show(HDC hdc) override;

};

class CubeShape : public RectShape, public LineShape {

public:

void Show(HDC hdc) override;

};

shape.cpp:

#include "shape.h"

#include <cmath>

void Shape::Set(long x1, long y1, long x2, long y2) {

```
    xs1 = x1;  
    ys1 = y1;  
    xs2 = x2;  
    ys2 = y2;  
}
```

```
void PointShape::Show(HDC hdc) {  
    HBRUSH hBrush = CreateSolidBrush(RGB(0, 0, 0));  
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);  
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));  
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);  
  
    Ellipse(hdc, xs2 - 2, ys2 - 2, xs2 + 2, ys2 + 2);  
  
    SelectObject(hdc, hOldPen);  
    SelectObject(hdc, hOldBrush);  
    DeleteObject(hPen);  
    DeleteObject(hBrush);  
}
```

```
void LineShape::Show(HDC hdc) {  
    MoveToEx(hdc, xs1, ys1, NULL);  
    LineTo(hdc, xs2, ys2);  
}
```

```
void RectShape::Show(HDC hdc) {  
    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc,  
    GetStockObject(NULL_BRUSH));  
    long rx = std::abs(xs2 - xs1);
```

```

    long ry = std::abs(ys2 - ys1);
    Rectangle(hdc, xs1 - rx, ys1 - ry, xs1 + rx, ys1 + ry);
    SelectObject(hdc, oldBrush);
}

void EllipseShape::Show(HDC hdc) {
    HBRUSH currentBrush = NULL;
    HBRUSH oldBrush = NULL;

    if (GetROP2(hdc) == R2_NOTXORPEN) {
        currentBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
    } else {
        currentBrush = CreateSolidBrush(RGB(255, 192, 203));
    }

    oldBrush = (HBRUSH)SelectObject(hdc, currentBrush);
    Ellipse(hdc, xs1, ys1, xs2, ys2);
    SelectObject(hdc, oldBrush);

    if (GetROP2(hdc) != R2_NOTXORPEN) {
        DeleteObject(currentBrush);
    }
}

void LineOOShape::Show(HDC hdc) {
    long origX1 = xs1, origY1 = ys1;
    long origX2 = xs2, origY2 = ys2;

    LineShape::Set(origX1, origY1, origX2, origY2);
}

```

```
LineShape::Show(hdc);
```

```
const long R = 5;
```

```
EllipseShape::Set(origX1 - R, origY1 - R, origX1 + R, origY1 + R);
```

```
EllipseShape::Show(hdc);
```

```
EllipseShape::Set(origX2 - R, origY2 - R, origX2 + R, origY2 + R);
```

```
EllipseShape::Show(hdc);
```

```
Shape::Set(origX1, origY1, origX2, origY2);
```

```
}
```

```
void CubeShape::Show(HDC hdc) {
```

```
    long origX1 = xs1, origY1 = ys1;
```

```
    long origX2 = xs2, origY2 = ys2;
```

```
    long dx = (origX2 - origX1) / 3;
```

```
    long dy = (origY2 - origY1) / 3;
```

```
    long f_left  = origX1,   f_top   = origY1;
```

```
    long f_right = origX2 - dx, f_bottom = origY2;
```

```
    long b_left  = f_left + dx, b_top   = f_top - dy;
```

```
    long b_right = f_right + dx, b_bottom = f_bottom - dy;
```

```
    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc,  
GetStockObject(NULL_BRUSH));
```

```
    Rectangle(hdc, f_left, f_top, f_right, f_bottom);
```

```
    Rectangle(hdc, b_left, b_top, b_right, b_bottom);
```

```
    SelectObject(hdc, oldBrush);
```

```

LineShape::Set(f_left, f_top, b_left, b_top);
LineShape::Show(hdc);
LineShape::Set(f_right, f_top, b_right, b_top);
LineShape::Show(hdc);
LineShape::Set(f_left, f_bottom, b_left, b_bottom);
LineShape::Show(hdc);
LineShape::Set(f_right, f_bottom, b_right, b_bottom);
LineShape::Show(hdc);

Shape::Set(origX1, origY1, origX2, origY2);
}

```

editor.h:

```

#pragma once #include <windows.h> #include <commctrl.h> #include "shape.h"

#define MAX_SHAPES 103

#define ID_OBJECT_POINT 32771 #define ID_OBJECT_LINE 32772 #define
ID_OBJECT_RECT 32773 #define ID_OBJECT_ELLIPSE 32774 #define
ID_OBJECT_LINEOO 32775 #define ID_OBJECT_CUBE 32776

class MyEditor { private: HWND hWnd; HWND hToolBar; Shape** pcshape; int
shapeCount; int currentTool; bool isDrawing; long startX, startY, endX, endY; Shape*
tempShape; HPEN hRubberPen;

Shape* CreateShapeFactory(int tool);

public: MyEditor(); ~MyEditor();

void Start(HWND h);
void CreateToolbar();
void OnInitMenuPopup(WPARAM wParam, LPARAM lParam);
LRESULT OnNotify(LPARAM lParam);

void OnLButtonDown(LPARAM lParam);
void OnMouseMove(LPARAM lParam);

```

```
void OnLButtonUp(LPARAM lParam);  
void OnPaint();  
void OnCommand(WPARAM wParam);
```

```
};
```

editor.cpp:

```
#include "editor.h"
```

```
MyEditor::MyEditor()
```

```
    : hWnd(NULL), hToolBar(NULL), shapeCount(0), currentTool(0),
```

```
    isDrawing(false), startX(0), startY(0), endX(0), endY(0),
```

```
    tempShape(nullptr), hRubberPen(nullptr)
```

```
{
```

```
    pcshape = new Shape*[MAX_SHAPES];
```

```
    for (int i = 0; i < MAX_SHAPES; i++) {
```

```
        pcshape[i] = nullptr;
```

```
    }
```

```
}
```

```
MyEditor::~MyEditor() {
```

```
    if (pcshape) {
```

```
        for (int i = 0; i < shapeCount; i++) {
```

```
            if (pcshape[i]) {
```

```
                delete pcshape[i];
```

```
                pcshape[i] = nullptr;
```

```
            }
```

```
        }
```

```
        delete[] pcshape;
```

```
        pcshape = nullptr;
```

```
}
```

```
if (tempShape) {  
    delete tempShape;  
    tempShape = nullptr;  
}
```

```
if (hRubberPen) {  
    DeleteObject(hRubberPen);  
    hRubberPen = nullptr;  
}  
}
```

```
void MyEditor::Start(HWND h) {  
    hWnd = h;  
    CreateToolbar();  
}
```

```
Shape* MyEditor::CreateShapeFactory(int tool) {  
    switch (tool) {  
        case 0: return new PointShape();  
        case 1: return new LineShape();  
        case 2: return new RectShape();  
        case 3: return new EllipseShape();  
        case 4: return new LineOOShape();  
        case 5: return new CubeShape();  
    }  
    return nullptr;  
}
```

```

void MyEditor::CreateToolbar() {

    InitCommonControls();

    HIMAGELIST hImageList = ImageList_Create(16, 16, ILC_COLOR24 |
    ILC_MASK, 6, 0);

    HDC hdcScreen = GetDC(hWnd);
    HDC hdcMem = CreateCompatibleDC(hdcScreen);
    HBITMAP hBmp = CreateCompatibleBitmap(hdcScreen, 96, 16);
    HBITMAP hOldBmp = (HBITMAP)SelectObject(hdcMem, hBmp);

    COLORREF maskColor = RGB(255, 0, 255);
    HBRUSH hBgBrush = CreateSolidBrush(maskColor);
    RECT rcFull = { 0, 0, 96, 16 };
    FillRect(hdcMem, &rcFull, hBgBrush);
    DeleteObject(hBgBrush);

    HPEN hBlackPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdcMem, hBlackPen);

    // 0: Точка
    HBRUSH hBlackBrush = CreateSolidBrush(RGB(0, 0, 0));
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdcMem, hBlackBrush);
    Ellipse(hdcMem, 6, 6, 11, 11);
    SelectObject(hdcMem, hOldBrush);
    DeleteObject(hBlackBrush);

    // 1: Лінія
    MoveToEx(hdcMem, 16 + 3, 12, NULL);

```

LineTo(hdcMem, 16 + 13, 3);

// 2: Прямокутник

hOldBrush = (HBRUSH)SelectObject(hdcMem, GetStockObject(NULL_BRUSH));

Rectangle(hdcMem, 32 + 3, 3, 32 + 13, 13);

SelectObject(hdcMem, hOldBrush);

// 3: Еліпс

HBRUSH hPinkBrush = CreateSolidBrush(RGB(255, 192, 203));

hOldBrush = (HBRUSH)SelectObject(hdcMem, hPinkBrush);

Ellipse(hdcMem, 48 + 2, 4, 48 + 14, 12);

SelectObject(hdcMem, hOldBrush);

DeleteObject(hPinkBrush);

// 4: Лінія з кружечками

MoveToEx(hdcMem, 64 + 4, 11, NULL);

LineTo(hdcMem, 64 + 12, 4);

hOldBrush = (HBRUSH)SelectObject(hdcMem, GetStockObject(NULL_BRUSH));

Ellipse(hdcMem, 64 + 2, 9, 64 + 6, 13);

Ellipse(hdcMem, 64 + 10, 2, 64 + 14, 6);

SelectObject(hdcMem, hOldBrush);

// 5: Куб

hOldBrush = (HBRUSH)SelectObject(hdcMem, GetStockObject(NULL_BRUSH));

Rectangle(hdcMem, 80 + 2, 6, 80 + 11, 14);

Rectangle(hdcMem, 80 + 5, 2, 80 + 14, 10);

MoveToEx(hdcMem, 80 + 2, 6, NULL); LineTo(hdcMem, 80 + 5, 2);

MoveToEx(hdcMem, 80 + 11, 6, NULL); LineTo(hdcMem, 80 + 14, 2);

MoveToEx(hdcMem, 80 + 2, 14, NULL); LineTo(hdcMem, 80 + 5, 10);

MoveToEx(hdcMem, 80 + 11, 14, NULL); LineTo(hdcMem, 80 + 14, 10);

SelectObject(hdcMem, hOldBrush);

SelectObject(hdcMem, hOldPen);

DeleteObject(hBlackPen);

SelectObject(hdcMem, hOldBmp);

DeleteDC(hdcMem);

ReleaseDC(hWnd, hdcScreen);

ImageList_AddMasked(hImageList, hBmp, maskColor);

DeleteObject(hBmp);

TBBUTTON tbb[6];

ZeroMemory(tbb, sizeof(tbb));

**tbb[0].iBitmap = 0; tbb[0].idCommand = ID_OBJECT_POINT; tbb[0].fsState =
TBSTATE_ENABLED | TBSTATE_CHECKED; tbb[0].fsStyle =
TBSTYLE_CHECKGROUP;**

**tbb[1].iBitmap = 1; tbb[1].idCommand = ID_OBJECT_LINE; tbb[1].fsState =
TBSTATE_ENABLED; tbb[1].fsStyle = TBSTYLE_CHECKGROUP;**

**tbb[2].iBitmap = 2; tbb[2].idCommand = ID_OBJECT_RECT; tbb[2].fsState =
TBSTATE_ENABLED; tbb[2].fsStyle = TBSTYLE_CHECKGROUP;**

**tbb[3].iBitmap = 3; tbb[3].idCommand = ID_OBJECT_ELLIPSE; tbb[3].fsState =
TBSTATE_ENABLED; tbb[3].fsStyle = TBSTYLE_CHECKGROUP;**

**tbb[4].iBitmap = 4; tbb[4].idCommand = ID_OBJECT_LINEOO; tbb[4].fsState =
TBSTATE_ENABLED; tbb[4].fsStyle = TBSTYLE_CHECKGROUP;**

**tbb[5].iBitmap = 5; tbb[5].idCommand = ID_OBJECT_CUBE; tbb[5].fsState =
TBSTATE_ENABLED; tbb[5].fsStyle = TBSTYLE_CHECKGROUP;**

**hToolBar = CreateWindowEx(
0, TOOLBARCLASSNAME, NULL,**

```

    WS_CHILD | WS_VISIBLE | WS_BORDER | TBSTYLE_TOOLTIPS,
    0, 0, 0, 0,
    hWnd, (HMENU)1001, GetModuleHandle(NULL), NULL
);

    SendMessage(hToolBar, TB_BUTTONSTRUCTSIZE,
(WPARAM)sizeof(TBBUTTON), 0);

    SendMessage(hToolBar, TB_SETIMAGELIST, 0, (LPARAM)hImageList);
    SendMessage(hToolBar, TB_ADDBUTTONS, (WPARAM)6, (LPARAM)&tbb);
    SendMessage(hToolBar, TB_AUTOSIZE, 0, 0);
}

void MyEditor::OnLButtonDown(LPARAM lParam) {
    if (shapeCount < MAX_SHAPES) {
        isDrawing = true;
        startX = endX = LOWORD(lParam);
        startY = endY = HIWORD(lParam);
        tempShape = CreateShapeFactory(currentTool);
        hRubberPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));
    }
}

void MyEditor::OnMouseMove(LPARAM lParam) {
    if (isDrawing && tempShape) {
        HDC hdc = GetDC(hWnd);
        SetROP2(hdc, R2_NOTXORPEN);
        SetBkMode(hdc, TRANSPARENT);

        HPEN oldPen = (HPEN)SelectObject(hdc, hRubberPen);

```

```
tempShape->Set(startX, startY, endX, endY);
```

```
tempShape->Show(hdc);
```

```
endX = LOWORD(IParam);
```

```
endY = HIWORD(IParam);
```

```
tempShape->Set(startX, startY, endX, endY);
```

```
tempShape->Show(hdc);
```

```
SelectObject(hdc, oldPen);
```

```
ReleaseDC(hWnd, hdc);
```

```
}
```

```
}
```

```
void MyEditor::OnLButtonUp(LPARAM IParam) {
```

```
    if (isDrawing) {
```

```
        isDrawing = false;
```

```
        pcshape[shapeCount] = CreateShapeFactory(currentTool);
```

```
        pcshape[shapeCount]->Set(startX, startY, endX, endY);
```

```
        shapeCount++;
```

```
        delete tempShape;
```

```
        tempShape = nullptr;
```

```
        if (hRubberPen) {
```

```
            DeleteObject(hRubberPen);
```

```
            hRubberPen = nullptr;
```

```
        }
```

```
        InvalidateRect(hWnd, NULL, TRUE);  
    }  
}
```

```
void MyEditor::OnPaint() {  
    PAINTSTRUCT ps;  
    HDC hdc = BeginPaint(hWnd, &ps);  
  
    for (int i = 0; i < shapeCount; i++) {  
        if (pcshape[i]) {  
            pcshape[i]->Show(hdc);  
        }  
    }  
  
    EndPaint(hWnd, &ps);  
}
```

```
void MyEditor::OnCommand(WPARAM wParam) {  
    int wParam = LOWORD(wParam);  
    switch (wParam) {  
        case ID_OBJECT_POINT:  currentTool = 0; break;  
        case ID_OBJECT_LINE:   currentTool = 1; break;  
        case ID_OBJECT_RECT:   currentTool = 2; break;  
        case ID_OBJECT_ELLIPSE: currentTool = 3; break;  
        case ID_OBJECT_LINEOO: currentTool = 4; break;  
        case ID_OBJECT_CUBE:   currentTool = 5; break;  
    }  
}
```

```

    if (hToolBar) {

        SendMessage(hToolBar, TB_CHECKBUTTON, ID_OBJECT_POINT +
currentTool, MAKELONG(TRUE, 0));

    }
}

void MyEditor::OnInitMenuPopup(WPARAM wParam, LPARAM lParam) {

    HMENU hMenu = (HMENU)wParam;

    for (int i = 0; i < 6; i++) {

        CheckMenuItem(hMenu, ID_OBJECT_POINT + i, MF_BYCOMMAND |
MF_UNCHECKED);

    }

    CheckMenuItem(hMenu, ID_OBJECT_POINT + currentTool, MF_BYCOMMAND |
MF_CHECKED);

}

LRESULT MyEditor::OnNotify(LPARAM lParam) {

    LPNMHDR pnmh = (LPNMHDR)lParam;

    if (pnmh->code == TTN_GETDISPINFOW) {

        LPTOOLTIPTEXTW lpttt = (LPTOOLTIPTEXTW)lParam;

        lpttt->hinst = NULL;

        static WCHAR strPoint[] = { 0x041A, 0x0440, 0x0430, 0x043F, 0x043A, 0x0430, 0
}; // Крапка

        static WCHAR strLine[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, 0 }; //
Лінія

        static WCHAR strRect[] = { 0x041F, 0x0440, 0x044F, 0x043C, 0x043E, 0x043A,
0x0443, 0x0442, 0x043D, 0x0438, 0x043A, 0 }; // Прямокутник

        static WCHAR strEllipse[] = { 0x0415, 0x043B, 0x0456, 0x043F, 0x0441, 0 }; //
Еліпс

```

```
static WCHAR strLineOO[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, ' ',
0x0437, ' ', 0x043A, 0x0440, 0x0443, 0x0436, 0x0435, 0x0447, 0x043A, 0x0430, 0x043C,
0x0438, 0 }; // Лінія з кружечками
```

```
static WCHAR strCube[] = { 0x041A, 0x0430, 0x0440, 0x043A, 0x0430, 0x0441, '
', 0x043A, 0x0443, 0x0431, 0x0430, 0 }; // Каркас куба
```

```
switch (lpttt->hdr.idFrom) {
    case ID_OBJECT_POINT: lpttt->lpszText = strPoint; break;
    case ID_OBJECT_LINE: lpttt->lpszText = strLine; break;
    case ID_OBJECT_RECT: lpttt->lpszText = strRect; break;
    case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipse; break;
    case ID_OBJECT_LINEOO: lpttt->lpszText = strLineOO; break;
    case ID_OBJECT_CUBE: lpttt->lpszText = strCube; break;
}
}
return 0;
}
```

Lab4.cpp:

```
#include "framework.h" #include "Resource.h" #include "editor.h"
```

```
#pragma comment(lib, "comctl32.lib")
```

```
HINSTANCE hInst; WCHAR szTitle[100] = L"Lab4 - Об'єктний редактор"; WCHAR
szWindowClass[100] = L"Lab4Class";
```

```
static MyEditor myEditor;
```

```
ATOM MyRegisterClass(HINSTANCE hInstance); BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
```

```
int APIENTRY wWinMain(In HINSTANCE hInstance, In_opt HINSTANCE hPrevInstance,
In LPWSTR lpCmdLine, In int nCmdShow) {
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);
```

```

MyRegisterClass(hInstance);

if (!InitInstance(hInstance, nCmdShow))
{
    return FALSE;
}

MSG msg;

while (GetMessage(&msg, nullptr, 0, 0))
{
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}

return (int)msg.wParam;

}

ATOM MyRegisterClass(HINSTANCE hInstance) { WNDCLASSEXW wcex;

wcex.cbSize = sizeof(WNDCLASSEX);

wcex.style          = CS_HREDRAW | CS_VREDRAW;
wcex.lpfnWndProc    = WndProc;
wcex.cbClsExtra     = 0;
wcex.cbWndExtra     = 0;
wcex.hInstance      = hInstance;
wcex.hIcon          = LoadIcon(nullptr, IDI_APPLICATION);
wcex.hCursor        = LoadCursor(nullptr, IDC_ARROW);
wcex.hbrBackground  = (HBRUSH)(COLOR_WINDOW + 1);
wcex.lpszMenuName   = MAKEINTRESOURCEW(IDC_LAB4);
wcex.lpszClassName  = szWindowClass;
wcex.hIconSm        = LoadIcon(nullptr, IDI_APPLICATION);

return RegisterClassExW(&wcex);

}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) { hInst = hInstance;

```

```
HWND hWnd = CreateWindowW(szWindowClass, szTitle,  
WS_OVERLAPPEDWINDOW,  
    CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance,  
nullptr);
```

```
if (!hWnd)  
{  
    return FALSE;  
}
```

```
ShowWindow(hWnd, nCmdShow);  
UpdateWindow(hWnd);
```

```
return TRUE;
```

```
}
```

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,  
LPARAM lParam) { switch (message) { case WM_CREATE: myEditor.Start(hWnd); break;
```

```
case WM_LBUTTONDOWN:  
    myEditor.OnLButtonDown(lParam);  
    break;
```

```
case WM_MOUSEMOVE:  
    myEditor.OnMouseMove(lParam);  
    break;
```

```
case WM_LBUTTONUP:  
    myEditor.OnLButtonUp(lParam);  
    break;
```

```
case WM_PAINT:  
    myEditor.OnPaint();  
    break;
```

```
case WM_COMMAND:  
    {  
        int wmId = LOWORD(wParam);  
        switch (wmId)  
        {  
            case IDM_EXIT:
```

```

        DestroyWindow(hWnd);
        break;
    default:
        myEditor.OnCommand(wParam);
        break;
    }
}
break;

case WM_INITMENUPOPUP:
    myEditor.OnInitMenuPopup(wParam, lParam);
    break;

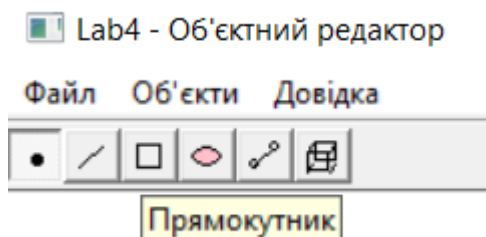
case WM_NOTIFY:
    return myEditor.OnNotify(lParam);

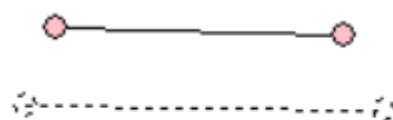
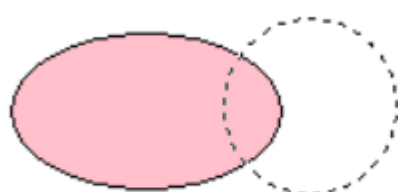
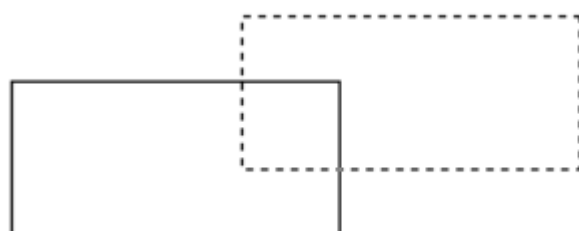
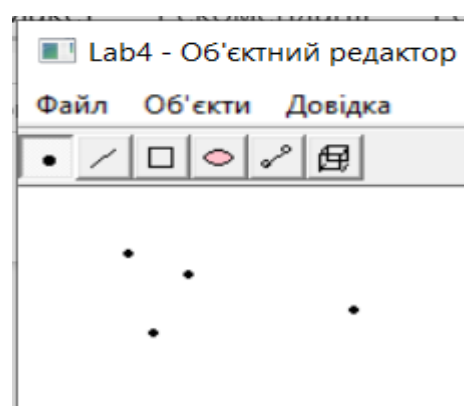
case WM_DESTROY:
    PostQuitMessage(0);
    break;

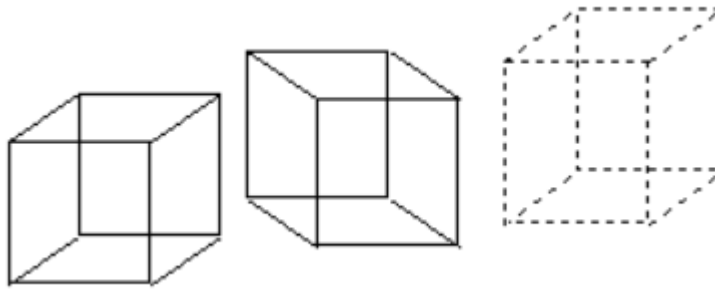
default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

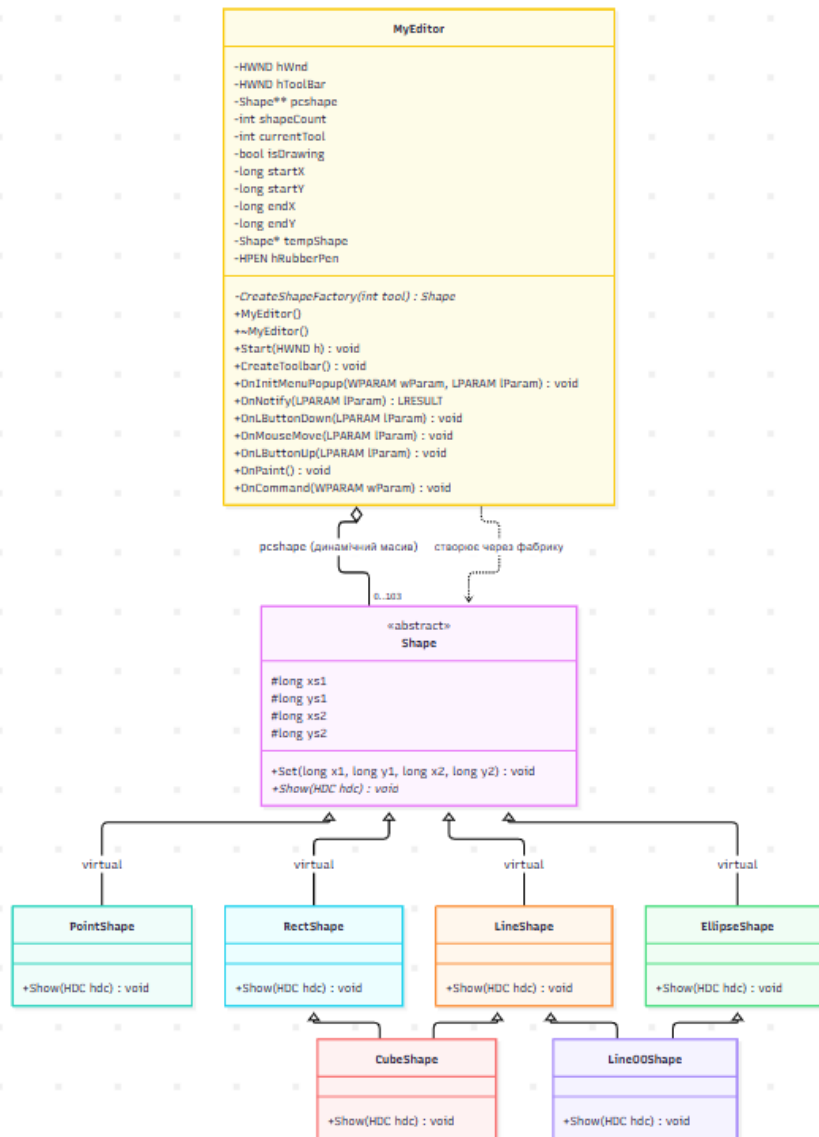
Скріншоти:







Діаграма класів:



Висновок:

Я отримав вміння та навички проєктування класів, виконавши модернізацію коду графічного редактора в об'єктно-орієнтованому стилі для забезпечення зручного додавання нових типів об'єктів.